

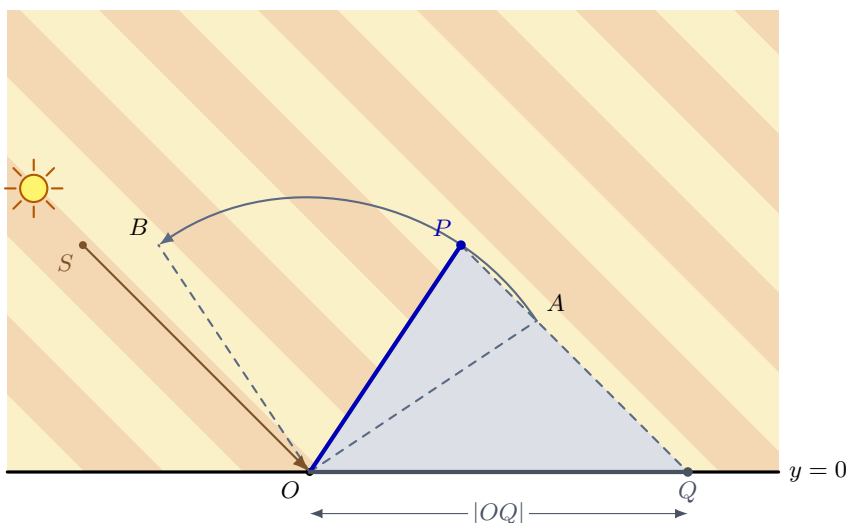
Problem A

夏日影

Time limit: 2 seconds

队伍的实力本来有很多维，一场比赛的排名却只是它在某个方向上的投影。赛场上，也许有些方向还能由你控制；但有时候，说不定只是巴库的阳光不太站在你这边……

二维平面上，一根定长直杆的一端铰接在地面上，可以在两个边界方向之间转动，且杆身始终位于地面上方。太阳光是沿固定方向传播的平行光。求杆在所有允许方向下影长的最小值和最大值。



图中， O 是原点， $\vec{SO}$ 表示太阳光的传播方向，杆可以从 $\vec{OA}$ 逆时针转动至 $\vec{OB}$ 。当前杆为 OP ，此时影长为 $|OQ|$ 。

Input

第一行包含一个整数 T ($1 \leq T \leq 10^5$)，表示测试数据的组数。接下来是各组测试数据的描述。

每组测试数据包含六个整数 s_x, s_y, a_x, a_y, b_x 和 b_y ，分别确定 $S = (s_x, s_y)$ 、 $A = (a_x, a_y)$ 和 $B = (b_x, b_y)$ ($-10^5 \leq s_x, a_x, b_x \leq 10^5$; $1 \leq s_y, a_y, b_y \leq 10^5$)。

保证 $|OA| = |OB|$ ，且 $\vec{OB}$ 位于 $\vec{OA}$ 的逆时针方向或与其重合。

Output

对于每组测试数据，输出两个实数，依次表示影长的最小值和最大值。

如果答案的绝对误差或相对误差不超过 10^{-6} ，则认为答案正确。

Sample Input 1

```
2
-3 3 3 2 -2 3
0 1 1 1 -1 1
```

Sample Output 1

```
1.0000000000000000 5.099019513592784
0.0000000000000000 1.0000000000000000
```

Problem B

老虎机

Time limit: 1 seconds

游戏厅里有 n 台老虎机，编号为 $1, 2, \dots, n$ 。游戏开始时，每台老虎机最初的奖金都是从区间 $[0, m]$ 中独立且均匀随机选取的一个 **实数**。你的作弊手段可以让你在每次选择前看到所有老虎机当前的奖金。

你的策略是选择当前奖金最多的老虎机：奖金较多的老虎机排名更高，奖金相同时编号较大的老虎机排名更高。领取奖金后，被选中的老虎机会独立地从 $[0, m]$ 中均匀随机生成一笔新的奖金。你的作弊手段会立刻告诉你新的数额，其他老虎机的奖金则保持不变。

求恰好玩 k 次后，获得的奖金总额的期望。

Input

第一行包含三个整数 n 、 m 和 k ($1 \leq n, k \leq 300$, $1 \leq m \leq 10^9$)。

Output

输出获得的 k 笔奖金总额的期望。如果答案的绝对误差或相对误差不超过 10^{-6} ，则认为答案正确。

Sample Input 1

1 10 3

Sample Output 1

15.0000000000

Sample Input 2

2 6 2

Sample Output 2

7.5000000000

在第一组样例中，机器只有一台。每次游玩获得的奖金期望为 5，因此奖金总额的期望为 15。在第二组样例中，前两次游玩获得的奖金期望分别为 4 和 3.5，因此奖金总额的期望为 7.5。

Problem C

老虎机 2

Time limit: 4 seconds

游戏厅里仍然有 n 台老虎机，编号为 $1, 2, \dots, n$ 。老板发现了你的作弊行为，于是修改了老虎机的程序。现在，游戏开始时，每台老虎机最初的奖金都是从 $0, 1, \dots, m$ 中独立且均匀随机选取的一个整数。不过，由于你的作弊手段高超，你仍然可以在每次选择前看到所有老虎机当前的奖金。

为了不再因为每次都选中最高奖金而露馅，你换了一种更聪明的策略：在第 i 次游玩时，你选择当前奖金第 a_i 多的老虎机：奖金较多的老虎机排名更高，奖金相同时编号较大的老虎机排名更高。领取奖金后，被选中的老虎机会独立地从整数 $0, 1, \dots, m$ 中均匀随机生成一笔新的奖金。你的作弊手段会立刻告诉你新的数额，其他老虎机的奖金则保持不变。

求恰好游玩 k 次后，获得的奖金总额的期望。

Input

第一行包含三个整数 n 、 m 和 k ($1 \leq n, k \leq 300$, $1 \leq m \leq 10^9$)。

第二行包含 k 个整数 $a_1, a_2, \dots, a_k$ ($1 \leq a_i \leq n$)。

Output

可以证明，精确答案可以写成最简分数 $\frac{p}{q}$ ，其中 $q \not\equiv 0 \pmod{10^9 + 7}$ 。输出整数 x ($0 \leq x < 10^9 + 7$)，满足 $x \cdot q \equiv p \pmod{10^9 + 7}$ 。

Sample Input 1

1 10 3
1 1 1

Sample Output 1

15

Sample Input 2

2 2 2
1 2

Sample Output 2

777777785

在第一组样例中，机器只有一台。每次游玩获得的奖金期望为 5，因此奖金总额的期望为 15。在第二组样例中，奖金总额的期望为 $\frac{16}{9}$ 。由于 $9^{-1} \equiv 111\,111\,112 \pmod{10^9 + 7}$ ，因此应输出 $16 \cdot 111\,111\,112 \pmod{10^9 + 7} = 777\,777\,785$ 。

Problem D

绝对多数

Time limit: 5 seconds

公园里有 n 个路口，由小径相连。任意两个路口之间，恰好有一条不重复经过路口的路线。

每个路口都要立一块写有小写英文字母的路牌。有些路牌已经写好，其余路牌上的字母用问号表示。你需要将每个问号替换为一个小写英文字母，同时不能改动已经写好的字母。

考虑任意一条经过至少三个路口的路线。如果某个字母在这条路线中出现的次数超过路口总数的一半，就称这个字母占绝对多数。如果一条不少于三个点的路线上同一个字母占了绝对多数，那么路人会感到困惑，因此这么做是不允许的。

请补全所有路牌，或判断不存在合法的写法。

Input

第一行包含一个整数 T ($1 \leq T \leq 1000$)，表示测试数据的组数。接下来是各组测试数据的描述。

每组测试数据的第一行包含一个整数 n ($3 \leq n \leq 5000$)，表示路口的数量。

每组测试数据的第二行包含一个长度为 n 的字符串 s 。 s 的第 i 个字符是一个小写英文字母或问号：小写英文字母表示路口 i 的路牌上已经写好的字母， $?$ 表示这个字母尚未写好。

接下来的 $n - 1$ 行中，第 i 行包含两个整数 u_i 和 v_i ($1 \leq u_i, v_i \leq n$)，表示路口 u_i 和 v_i 之间有一条小径。

保证每组测试数据中给定的边构成一棵树。

保证所有测试数据中 n 的总和不超过 5000。

Output

对于每组测试数据，输出一个合法的标记方案，具体格式如下。

如果无解，输出 -1 。

否则，输出一个长度为 n 的字符串，其第 i 个字符表示写在路口 i 的路牌上的字母。如果有多个解，输出任意一个即可。

Sample Input 1

```
2
5
a?c??
1 2
2 3
2 4
4 5
5
aab?a
1 2
2 3
3 4
4 5
```

Sample Output 1

```
abcda
-1
```

Problem E

璀璨宝石

Time limit: 2 seconds

在桌游《璀璨宝石》(Splendor)中, 玩家通过拿取不同颜色的宝石来发展自己的商业帝国。原版规则较为复杂, 因此本题仅采用下文描述的简化规则。原桌游中的其他规则均不适用于本题。

共有五种颜色的宝石, 每种颜色的宝石数量均无限。你的目标是恰好拿到 a_i 枚颜色 i 的宝石。

每个回合, 你必须恰好执行以下一种操作:

- 分别拿取三种不同颜色的宝石各一枚;
- 拿取两枚颜色相同的宝石。

任何颜色的宝石都不能拿取超过目标数量。求达成目标所需的最少回合数, 或判断目标无法达成。

Input

第一行包含一个整数 T ($1 \leq T \leq 10^5$), 表示测试数据的组数。接下来是各组测试数据的描述。

每组测试数据包含五个整数 $a_1, a_2, \dots, a_5$ ($0 \leq a_i \leq 10^9$), 其中 a_i 表示需要拿取的颜色 i 的宝石数量。

Output

对于每组测试数据, 输出一个整数, 表示恰好拿到所需宝石的最少回合数。如果无解, 输出 -1 。

Sample Input 1

3	2
2 2 2 0 0	2
3 1 1 0 0	0
0 0 0 0 0	

Sample Output 1

Sample Input 2

2	-1
1 0 0 0 0	2
2 1 1 1 1	

Sample Output 2

在样例一的第一组测试数据中, 两个回合都可以分别拿取颜色 1、2、3 的宝石各一枚。

在样例一的第二组测试数据中, 第一个回合可以分别拿取颜色 1、2、3 的宝石各一枚, 第二个回合可以拿取两枚颜色 1 的宝石。

在样例二的第一组测试数据中, 两种操作都无法只拿取一枚宝石。

在样例二的第二组测试数据中, 第一个回合可以分别拿取颜色 1、2、3 的宝石各一枚, 第二个回合可以分别拿取颜色 1、4、5 的宝石各一枚。

Problem F

出现次数

Time limit: 4 seconds

你正在为一篇由小写字符串 S 表示的文稿制作索引。 S 的长度为 n ，字符下标从 1 到 n 。

对于任意非空字符串 T ，定义 $\text{occ}(T)$ 为满足 $1 \leq p \leq n - |T| + 1$ 以及下式的起始位置 p 的数量：

$$S[p \dots p + |T| - 1] = T.$$

不同的出现可以重叠。

每次询问给出一个区间 $[l, r]$ 。考虑所有满足 $l \leq i \leq j \leq r$ 的非空子串 $S[i \dots j]$ 。这次询问的答案为

$$\sum_{l \leq i \leq j \leq r} \text{occ}(S[i \dots j]).$$

Input

第一行包含一个整数 n ($1 \leq n \leq 2 \times 10^5$)。

第二行包含字符串 S ，其长度为 n ，且仅由小写英文字母组成。

第三行包含一个整数 q ($1 \leq q \leq 2 \times 10^5$)。

接下来的 q 行中，每行包含两个整数 l 和 r ($1 \leq l \leq r \leq n$)。

Output

对于每次询问，输出一个整数，表示上文定义的和。

Sample Input 1

```
4
abab
3
1 2
1 4
2 3
```

Sample Output 1

```
6
16
5
```

Sample Input 2

```
4
aaaa
3
1 1
1 2
2 4
```

Sample Output 2

```
4
11
20
```

在第一组样例的第一次询问中，求和中的三个子串为 $S[1 \dots 1] = a$ 、 $S[2 \dots 2] = b$ 和 $S[1 \dots 2] = ab$ 。它们在 S 中的出现次数分别为 2、2 和 2，因此答案为 6。

在第二组样例的第二次询问中， $S[1 \dots 1]$ 和 $S[2 \dots 2]$ 均为 a ，各自贡献 $\text{occ}(a) = 4$ ； $S[1 \dots 2] = aa$ ，贡献 $\text{occ}(aa) = 3$ 。因此答案为 $4 + 4 + 3 = 11$ 。

Problem G

还是 A+B 问题

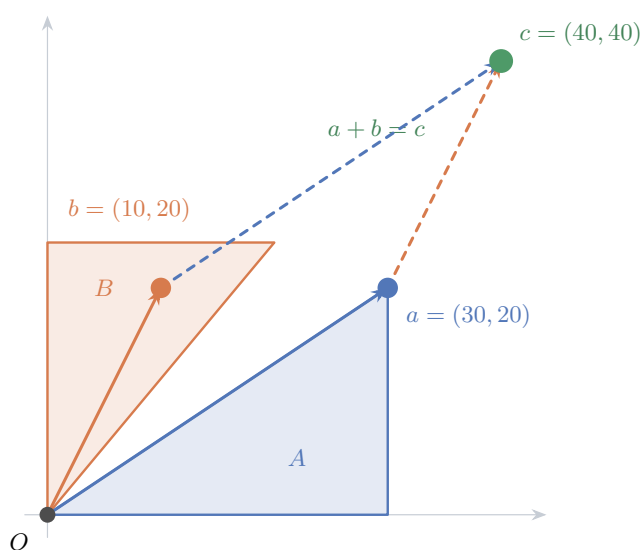
Time limit: 10 seconds

给定两个可能退化的简单多边形 A 和 B 。每个多边形都包含其内部和边界；退化的多边形可以是线段或点。

对于两个点 $a = (x_a, y_a)$ 和 $b = (x_b, y_b)$ ，定义它们的和为

$$a + b = (x_a + x_b, y_a + y_b).$$

对于每个询问点 c_i ，判断是否存在 $a \in A$ 和 $b \in B$ ，满足 $a + b = c_i$ 。如果存在，给出任意一组这样的点。



第一个样例第一组测试数据的示意图。

Input

第一行包含一个整数 T ($1 \leq T \leq 200$)，表示测试数据的组数。接下来是各组测试数据的描述。

每组测试数据的第一行包含三个整数 n 、 m 和 q ($1 \leq n, m, q \leq 200$)，分别表示 A 的顶点数、 B 的顶点数和询问数。

第二行包含 $2n$ 个整数 $x_{a_1}, y_{a_1}, x_{a_2}, y_{a_2}, \dots, x_{a_n}, y_{a_n}$ ，按逆时针顺序描述 A 的各个顶点。

第三行包含 $2m$ 个整数 $x_{b_1}, y_{b_1}, x_{b_2}, y_{b_2}, \dots, x_{b_m}, y_{b_m}$ ，按逆时针顺序描述 B 的各个顶点。

接下来的 q 行中，第 i 行包含两个整数 x_{c_i} 和 y_{c_i} ，表示询问点 c_i 的坐标。

所有输入坐标均在 0 到 2×10^6 之间。每个多边形内给出的顶点互不相同。只有一个顶点的多边形表示这个点，只有两个顶点的多边形表示连接这两个点的线段；顶点数不少于三个的多边形是内部连通且具有正面积的简单多边形，并且任意三个循环相邻的顶点均不共线。

保证所有测试数据满足 $\sum n \leq 500$ 且 $\sum m \leq 500$ 。请注意，所有测试数据中 q 的总和无额外限制。

Output

对于每个询问，输出一行。

如果有解，输出四个实数 x_a 、 y_a 、 x_b 和 y_b ，表示任意一组满足 $a = (x_a, y_a) \in A$ 、 $b = (x_b, y_b) \in B$ 且 $a + b = c_i$ 的点。否则，输出 $-1 \ -1 \ -1 \ -1$ 。

如果输出的两个点到对应多边形的欧几里得距离均不超过 10^{-6} ，且 $a + b$ 到 c_i 的欧几里得距离不超过 10^{-6} ，则这组答案会被接受。

Sample Input 1

```

4
3 3 3
0 0 30 0 30 20
0 0 20 24 0 24
40 40
31 0
31 2
2 2 3
0 0 0 300
0 0 300 0
1 2
2 1
301 0
3 4 2
0 0 1 0 0 1
0 0 1 0 1 1 0 1
2 1
1 2
8 3 3
0 0 12 6 3 6 7 4 1 1 1 5 2 6 0 6
0 0 2 0 0 1
2 7
5 5
4 5
  
```

Sample Output 1

```

30 20 10 20
-1 -1 -1 -1
29.5 0 1.5 2
0 2 1 0
0 1 2 0
-1 -1 -1 -1
1 0 1 1
0 1 1 1
2 6 0 1
5 5 0 0
-1 -1 -1 -1
  
```

Sample Input 2

```

1
1 1 1
2 3
4 5
6 8
  
```

Sample Output 2

```

2 3 4 5
  
```

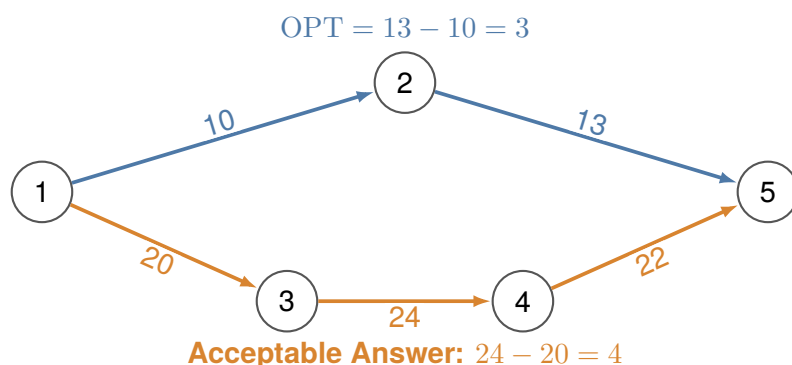

Problem H

差不多得了

Time limit: 4 seconds

“差不多得了”物流公司正在规划一条货运路线。公司的运输网络由路口和单向道路组成，每条道路都有一个整数限速。为了避免货车一路上反复加速和减速，公司希望所选路线上的限速尽量接近。一条路径的极差是这条路径上的最高限速减去最低限速。

给定两个路口 s 和 t ，令 OPT 表示从 s 到 t 的所有路径中，极差的最小值。秉承“差不多得了”的精神，你不一定要找到最优解，只需给出一个 $\sqrt{2}$ 近似解：一条从 s 到 t 的路径，其极差不超过 $\sqrt{2} \cdot OPT$ 。



第一组样例：蓝色路径的极差为 $13 - 10 = 3 = OPT$ 。
 橙色路径的极差为 $24 - 20 = 4$ ，且 $4 \leq \sqrt{2} \cdot 3$ ，因此也是可接受的答案。

Input

第一行包含四个整数 n, m, s 和 t ($2 \leq n \leq 10^5$, $1 \leq m \leq 2 \times 10^5$, $1 \leq s, t \leq n$, $s \neq t$)，分别表示路口数、道路数、起点和终点。

接下来的 m 行中，第 i 行包含三个整数 u_i, v_i 和 w_i ($1 \leq u_i, v_i \leq n$, $1 \leq w_i \leq 10^9$)，表示一条从路口 u_i 指向路口 v_i 、限速为 w_i 的单向道路。

保证给出的图没有自环或重边，且存在一条从 s 到 t 的路径。

Output

第一行输出一个整数 k ($2 \leq k \leq n$)，表示路径上的顶点数。

第二行输出 k 个互不相同的整数 $p_1, p_2, \dots, p_k$ ($1 \leq p_i \leq n$)，依次表示路径上的顶点。它们必须满足 $p_1 = s, p_k = t$ ，并且对于每个 $1 \leq i < k$ ，都存在一条从 p_i 指向 p_{i+1} 的边。

令 ANS 表示输出路径的极差。若 $ANS^2 \leq 2 \cdot OPT^2$ ，则认为答案正确。如果有多个解，输出任意一个即可。

Sample Input 1

```
5 5 1 5
1 2 10
2 5 13
1 3 20
3 4 24
4 5 22
```

Sample Output 1

```
3
1 2 5
```

Sample Input 2

```
4 5 1 4
1 2 3
2 4 8
1 3 5
3 4 7
2 3 6
```

Sample Output 2

```
3
1 3 4
```

Problem I

选择用户名

Time limit: 5 seconds

猪标最近（在 AI 的一点小小帮助下）写了一个用户管理系统。这个系统的数据库其实就是一个直接存储在硬盘上的文件。系统会把查询的用户名当作前缀：搜索一个字符串时，所有用户名以该字符串开头的用户都会被找到。

因此，如果一个用户名是另一个用户名的前缀，精确查找较短的用户名时也会找到较长的用户名。猪标想起了无前缀码，决定对用户名提出相同的要求。如果对于任意两个不同的用户，其中任何一个用户的名都不是另一个用户名的前缀，那么这些用户名构成一个无前缀集合。一个字符串也是其自身的前缀，因此两个用户不能使用相同的用户名。

系统中有 n 个已有用户。第 i 个用户提供两个候选用户名：左侧候选名 l_i 和右侧候选名 r_i 。你需要为每个用户恰好选择一个候选名，使所有选出的用户名构成一个无前缀集合；或者判断这无法做到。

Input

第一行包含一个整数 T ($1 \leq T \leq 10^4$)，表示测试数据的组数。接下来是各组测试数据的描述。

每组测试数据的第一行包含一个整数 n ($1 \leq n \leq 5 \times 10^5$)。

接下来的 n 行中，第 i 行包含两个非空字符串 l_i 和 r_i ，表示第 i 个用户的左侧候选名和右侧候选名。每个字符串仅由小写英文字母组成。

保证所有测试数据中 n 的总和不超过 5×10^5 。

保证所有测试数据中全部字符串的长度之和不超过 10^6 。

Output

对于每组测试数据，如果答案存在，输出 YES；否则输出 NO。

如果答案存在，下一行输出一个长度为 n 、仅由字符 l 和 r 组成的字符串 s 。如果 s_i 为 l，则为第 i 个用户选择 l_i ；如果 s_i 为 r，则为第 i 个用户选择 r_i 。

如果有多个解，输出任意一个即可。

Sample Input 1

Sample Output 1

3	YES
3	lll
alice al	NO
bob bobby	YES
carol cat	lrr
2	
a ab	
a ab	
3	
a dog	
ab cat	
abc cow	

Sample Input 2**Sample Output 2**

2	YES
1	l
x x	YES
4	lrrr
a za	
ab zb	
abc zc	
abcd zd	

在第一个样例的第一组测试数据中，选出的用户名是 `alice`、`bob` 和 `carol`，其中任何一个都不是另一个的前缀。第二组测试数据中，任意选择出的两个用户名要么相同，要么分别为 `a` 和 `ab`，因此不存在合法方案。

在第二个样例中，只有一个用户的测试数据一定有解。在第二组测试数据中，选择 `za`、`zb`、`zc` 和 `zd` 可以得到一个无前缀集合。

Problem J

兜风

Time limit: 2 seconds

你正在规划穿行于一座城市的兜风路线。城市中有 n 个路口和 m 条双向道路。

一趟兜风从某个路口出发，经过至少 3 条道路后回到起点。途中不能重复经过任何道路，也不能重复经过除起点外的任何路口。因此，每趟兜风都沿着一个简单环行驶。

对于一个固定的路口 i ，选出尽可能多的经过 i 的兜风路线，并要求任意一条道路至多属于一条选中的路线。对于每个 $i = 1, 2, \dots, n$ ，求最多可以选出多少条路线。

Input

第一行包含一个整数 T ($1 \leq T \leq 2 \times 10^5$)，表示测试数据的组数。接下来是各组测试数据的描述。

每组测试数据的第一行包含两个整数 n 和 m ($1 \leq n \leq 2 \times 10^5$, $0 \leq m \leq 2 \times 10^5$)，分别表示路口数和道路数。

接下来的 m 行中，每行包含两个整数 u 和 v ($1 \leq u, v \leq n$, $u \neq v$)，表示路口 u 与路口 v 之间有一条双向道路。

路口编号为 1 到 n 。保证没有道路连接同一个路口，也没有两条道路连接同一对路口。城市不保证连通。

保证所有测试数据中 n 的总和不超过 2×10^5 ，且所有测试数据中 m 的总和不超过 2×10^5 。

Output

对于每组测试数据，输出 n 个整数，相邻两个整数之间用一个空格分隔。第 i 个整数表示最多可以选出多少条经过路口 i 的兜风路线，并且任意一条道路至多属于其中一条路线。

Sample Input 1

```
3
5 6
1 2
2 3
3 1
1 4
4 5
5 1
5 6
1 2
2 5
1 3
3 5
1 4
4 5
4 2
1 2
3 4
```

Sample Output 1

```
2 1 1 1 1
1 1 1 1 1
0 0 0 0
```

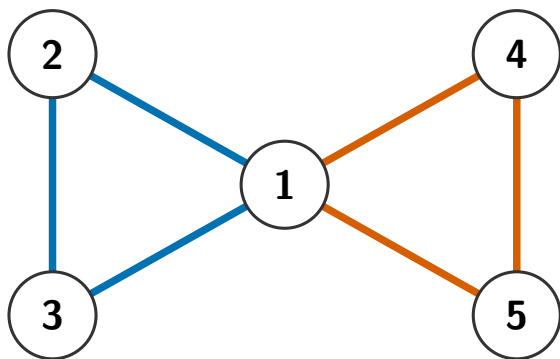
在第一组测试数据中，可以选出两条经过路口 1 且不共用道路的兜风路线： $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ 和 $1 \rightarrow 4 \rightarrow 5 \rightarrow 1$ 。因此，路口 1 的答案至少为 2。每条经过路口 1 的路线都要占用两条与它相连的道路，而这样的道路一共只有四条，所以不可能再选出第三条路线。路口 1 的答案恰好为 2。

路口 2 到 5 都在图中的某条兜风路线上，所以它们的答案至少为 1。另一方面，每个路口都只连着两条道

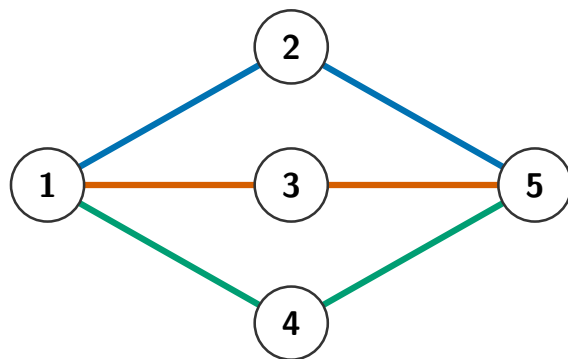
路；任何经过它的兜风路线都会同时用到这两条道路，因而无法再选出第二条不共用道路的路线。所以它们的答案都是 1。

在第二组测试数据中，路口 1 与路口 5 之间有三条不共用道路的路径，分别经过路口 2、3 和 4。任何经过路口 1 的兜风路线，都要从这三条路径中选出两条来绕成一圈。任意两个这样的环都会共用至少一条路径，因此不能同时选中。路口 1 的答案只能是 1。

图中每个路口都在至少一个环上，但没有任何路口能同时属于两条不共用道路的兜风路线，因此五个路口的答案都是 1。



第一组测试数据中的两条不共用道路的路线。



第二组测试数据中从路口 1 到路口 5 的三条路径。

左图中，蓝色路线为 $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ ，橙色路线为 $1 \rightarrow 4 \rightarrow 5 \rightarrow 1$ 。它们都经过路口 1，但没有共用道路。右图中，三种颜色分别表示从路口 1 到路口 5 的三条路径。

Problem K

组队

Time limit: 2 seconds

新学期， $3n$ 名新生来到校队试训。正式参加 ICPC/CCPC 之前，他们得先组成队伍。组队必须满足两条规则：

- 每支队伍恰好有三名队员；
- 每名队员恰好加入一支队伍。

规则只有两条，麻烦却不少：要组队，更要组对。每两名队员之间都有一个整数默契值；如果两个人实在合不来，这个值甚至可以是负数。一支队伍的得分是队内三对队员之间的默契值之和。

求所有合法组队方案中，队伍总得分的最大值。

Input

第一行包含一个整数 n ($1 \leq n \leq 8$)。

接下来的 $3n$ 行中，第 i 行包含 $3n$ 个整数 $a_{i,1}, a_{i,2}, \dots, a_{i,3n}$ ($-10^8 \leq a_{i,j} \leq 10^8$)。其中 $a_{i,j}$ 表示队员 i 和 j 之间的默契值。

保证对于所有 $1 \leq i, j \leq 3n$ ，都有 $a_{i,i} = 0$ 且 $a_{i,j} = a_{j,i}$ 。

Output

输出所有队伍得分之和的最大可能值。

Sample Input 1

2 0 5 4 -1 -2 -3 5 0 3 -2 -2 -3 4 3 0 -1 -4 -2 -1 -2 -1 0 7 6 -2 -2 -4 7 0 5 -3 -3 -2 6 5 0	30
---	----

Sample Output 1

Sample Input 2

1 0 -5 2 -5 0 1 2 1 0	-2
--------------------------------	----

Sample Output 2

在第一组样例中，一种最优分组是 (1, 2, 3) 和 (4, 5, 6)。总分为 $(5 + 4 + 3) + (7 + 6 + 5) = 30$ 。

在第二组样例中，三名队员只能组成唯一的队伍，其得分为 $-5 + 2 + 1 = -2$ 。

Problem L

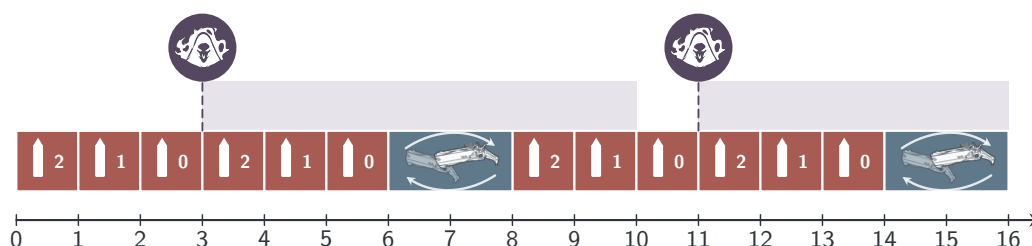
死神

Time limit: 2 seconds

死神想知道，如果战斗永远持续下去，他最多能以多快的速度使用手中的地狱火霰弹枪射击。

这件武器装满时共有 m 发子弹。射出一发子弹会消耗一发子弹，并花费恰好一秒。死神可以在两次射击之间装弹：他会扔掉手中的武器，再取出一件装有 m 发子弹的新武器。这个过程耗时 r 秒。

死神还可以使用幽灵形态，但不能在射击或装弹的过程中使用。使用幽灵形态不耗费时间，并会立即将这件武器的弹量恢复到 m 发。战斗开始时，幽灵形态已经可以使用。每次使用后，幽灵形态会进入 c 秒的冷却时间，在此期间无法使用。无论死神正在射击、装弹还是等待，幽灵形态都会正常冷却。



上图展示了样例中的第一组测试数据。可以证明，两次相邻装弹之间的最优操作会不断重复，每八秒射出六发子弹，因此答案为 $6/8 = 0.75$ 。

死神可以自行决定何时射击、装弹、使用幽灵形态或等待。假设战斗开始时武器已经装满，并且幽灵形态可以使用。令 $f(n)$ 表示死神在前 n 秒内最多能够完成的射击次数。可以证明，下面的极限存在：

$$\lim_{n \rightarrow \infty} \frac{f(n)}{n}.$$

求这个极限，也就是死神每秒最多能够射出的子弹数。

Input

第一行包含一个整数 T ($1 \leq T \leq 100$)，表示测试数据的组数。接下来是各组测试数据的描述。

每组测试数据包含三个整数 m 、 r 和 c ($1 \leq m, r, c \leq 100$)，分别表示武器装满时的子弹数、装弹所需的秒数和幽灵形态的冷却秒数。

Output

对于每组测试数据，输出死神的最大有效射速。

如果答案的绝对误差或相对误差不超过 10^{-9} ，则认为答案正确。

Sample Input 1

Sample Output 1

5	0.75000000000000
3 2 7	0.66666666666667
1 1 2	0.66666666666667
2 100 3	0.25000000000000
1 100 4	0.66666666666667
4 100 6	

Problem M

括号

Time limit: 2 seconds

给定两个长度为 n 的合法括号序列 S 和 T ，以及两个数组 a 和 b 。

如果可以在一个括号序列的字符之间插入 1 和 $+$ ，使其变成一个合法的算术表达式，则称这个括号序列是合法的。

长度为 n 的括号序列 P 的权重定义如下：

- 对于每个满足 $P_i = \text{[(]}$ 的位置 i ，加上 a_i ；
- 对于每个满足 $P_i = \text{[)]}$ 的位置 i ，加上 b_i 。

换言之，

$$\text{weight}(P) = \sum_{P_i = \text{[(]}} a_i + \sum_{P_i = \text{[)]}} b_i.$$

对于从 0 到 n 的每个整数 k ，选择恰好 k 个位置，并将每个选中位置 i 上的 S_i 替换为 T_i 。

记替换后得到的序列为 S' 。在所有恰好选择 k 个位置，且 S' 仍然是合法括号序列的方案中，求 S' 权重的最大可能值。

如果对于某个 k 无法得到合法括号序列，则该 k 的答案为 -1 。

Input

第一行包含一个整数 T ($1 \leq T \leq 250\,000$)，表示测试数据的组数。接下来是各组测试数据的描述。

每组测试数据包含五行。

每组测试数据的第一行包含一个偶数 n ($2 \leq n \leq 5 \times 10^5$)，表示括号序列的长度。

第二行包含一个长度为 n 的合法括号序列 S ，其中只包含 [(] 和 [)] 。

第三行包含一个长度为 n 的合法括号序列 T ，其中只包含 [(] 和 [)] 。

第四行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$)。

第五行包含 n 个整数 $b_1, b_2, \dots, b_n$ ($1 \leq b_i \leq 10^9$)。

保证所有测试数据中 n 的总和不超过 5×10^5 。

Output

对于每组测试数据，输出 $n + 1$ 个整数，相邻两个整数之间用一个空格分隔。

对于 $0 \leq k \leq n$ ，第 $(k + 1)$ 个整数应为恰好选择 k 个位置，并将所有选中位置上的 S_i 替换为 T_i 后，所得合法括号序列权重的最大可能值。

如果对于某个 k 无法得到合法括号序列，则第 $(k + 1)$ 个整数必须为 -1 。

Sample Input 1

```

5
2
()
()
1 2
3 4
6
((()))
()()()
1 2 3 4 5 6
6 5 4 3 2 1
4
()()
(())
1 5 4 6
2 2 5 5
8
(())()()
()()()()
8 5 1 4 2 6 7 3
1 9 7 6 6 5 9 2
10
(())()()()
(())()()()
2 3 4 1 9 5 8 7 1 10
1 9 6 7 6 2 8 5 3 10
  
```

Sample Output 1

```

5 5 5
12 12 18 18 18 18 18
12 12 16 16 16
42 42 45 45 47 47 47 45 45
48 48 52 52 53 53 53 53 53 53
  
```

在第一组测试数据中，替换任意选中位置的字符都不会改变序列。它的权重为 $a_1 + b_2 = 1 + 4 = 5$ 。

在第二组测试数据中， S 和 T 只有位置 2 和 5 不同。只替换其中一个位置会使序列不合法；同时替换这两个位置会将序列变为 T ，并使其权重从 12 增加到 18。其余四个位置即使被选中也不会改变序列。